

3.8

Remembering Things

Functions to Save and Load Files

THE PLAN

WHAT THIS LECTURE DOES

Takes the raw file I/O from the last lecture and wraps it in functions, then uses them properly:

1. **Load** whatever friends are already saved
2. **Show** them
3. Let the user **add more**
4. **Save** the combined list back

The friends list grows a little more every time you run the program.

LOADING

WRAPPING FILE I/O IN A FUNCTION



```
std::vector<std::string> loadFriendsFromFile(const std::string& fileName) {  
    std::vector<std::string> friends;  
    std::ifstream inputFile(fileName);  
  
    std::string friendName;  
    while (std::getline(inputFile, friendName)) {  
        friends.push_back(friendName);  
    }  
  
    return friends;  
}
```

SAVING

THE OTHER HALF



```
void saveFriendsToFile(const std::vector<std::string>& friends,
                      const std::string& fileName) {
    std::ofstream outputFile(fileName);
    for (const std::string& friendName : friends) {
        outputFile << friendName << "\n";
    }
    std::println("Saved {} friend(s) to {}", friends.size(), fileName);
}
```

LOAD FIRST

LOAD FIRST, THEN COLLECT MORE



```
std::vector<std::string> friends = loadFriendsFromFile(fileName);

std::println("\nYou already have {} friend(s) saved:", friends.size());
for (const std::string& friendName : friends) {
    std::println(" - {}", friendName);
}

std::vector<std::string> newFriends = collectMoreFriendNames();
friends.insert(friends.end(), newFriends.begin(), newFriends.end());

saveFriendsToFile(friends, fileName);
```

WHAT THIS PROGRAM ACTUALLY NEEDED

NEED	C++ BUILDING BLOCK
Say something to the screen	Output
Do a named, reusable action	A function
Ask the user something	Input
Compute something new	Arithmetic
Follow a different path	<code>if</code> / <code>else</code>
Repeat until told to stop	A loop
Hold a growing list	<code>std::vector</code>
Remember across runs	File I/O

IN YOUR TOOLBOX

- `std::print` / `std::println` - a single `{}`-style format string instead of chained `std::cout <<` - readable from lecture one
- **Brace initialization** (`{}`) - `int age{};` It makes your code safer
- `std::vector` - a ready-made collection that grows as you add to it
- **File I/O** - `std::ifstream` and `std::ofstream` for reading and writing files
- **Functions** - named, reusable actions that can take input and return output
- **Decisions and Loops** - `if` / `else` and `while` let your program follow different paths and repeat actions